

1 Wprowadzenie

2 Warstwa konwolucyjna

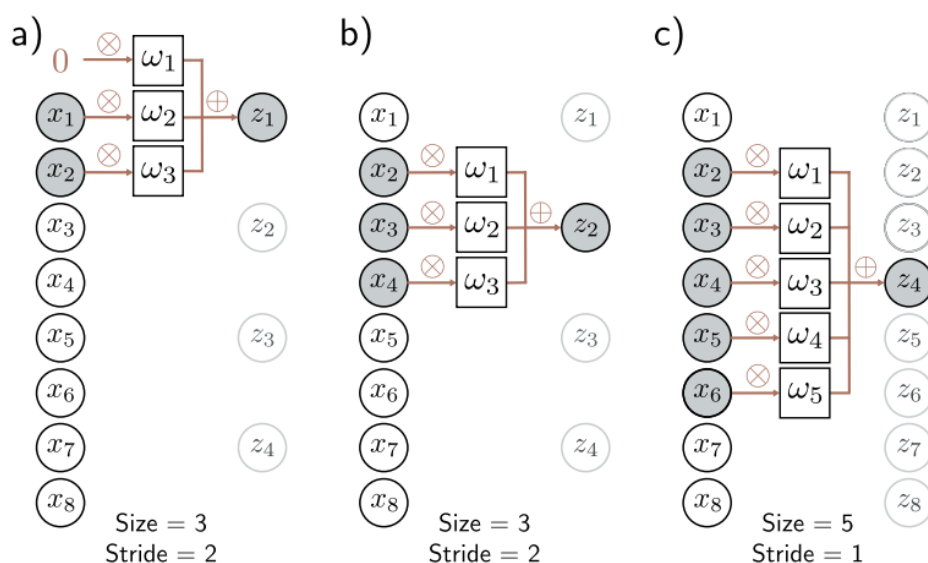
Warstwa konwolucyjna jest podstawowym budulcem modeli, które są wykorzystywane przez nas w tym projekcie: ResNet-ów (eng. Residual Networks) oraz sieci InceptionTime. Wykorzystywana jest ona przede wszystkim do pracy nad obrazami, ale może być również stosowana przy przetwarzaniu sygnałów, takich jak sygnały EKG badane przez nas. Warstwy te posiadają wiele zalet, które sprawiają, że są one idealnym wyborem do pracy z naszymi danymi. Przede wszystkim, w przeciwieństwie do warstw gęstych (warstwy w sieciach MLP), pozwalają one na wzięcie pod uwagę lokalnych zależności danych wejściowych, co jest kluczowe przy pracy z sygnałem takim jak EKG. Ponadto, warstwy te mają znacznie mniej parametrów, co pozwala na szybsze uczenie, a także zmniejsza ryzyko przeuczenia modelu.

Warstwa konwolucyjna opiera się jak sama nazwa wskazuje na operacji konwolucji. W jedynym wymiarze, operacja ta polega na przekształceniu wektora wejściowego x w wektor wyjściowy y , tak że każdy element y_i jest obliczany jako iloczyn skalarny fragmentu wektora wejściowego x z wektorem wag w , który jest przesuwany po całym wektorze wejściowym. Wzór na obliczenie elementu y_i wygląda następująco:

$$y_i = \sum_{j=0}^{k-1} w_j x_{i+j} \quad (1)$$

Gdzie $[w_0, w_1, \dots, w_{k-1}]^T$ to wektor wag, a k to rozmiar jądra konwolucyjnego (ang. kernel size).

Spójrzmy sobie teraz na parametry warstwy konwolucyjnej, które będziemy wykorzystywać w dalszej części.



Rysunek 1: Parametry warstwy konwolucyjnej. Źródło: adaptacja rysunku z [2, s. 164].

- **Kernel size** - rozmiar jądra konwolucyjnego, czyli długość wektora wag, który jest przesuwany po wektorze wejściowym. Różna długość jądra pozwala na wychwycenie bardziej lokalnych (mały rozmiar jądra) lub bardziej globalnych (duży rozmiar jądra) zależności
- **Stride** - ten parametr określa o ile pozycji wektora wejściowego przesuwane jest jądro konwolucyjne podczas obliczania kolejnych elementów wektora wyjściowego
- **Padding** - ten parametr określa, czy i ile zer jest dodawanych do wektora wejściowego na początku i końcu. Pozwala on zachować rozmiar wektora wyjściowego równy rozmiarowi wektora wejściowego (bo zarówno kernel size, jak i stride mogą powodować zmniejszenie tego rozmiaru)

- **Channels (In oraz Out)** - parametr ten określa liczbę kanałów wejściowych oraz wyjściowych. Liczba kanałów wejściowych określa, ile różnych wektorów wejściowych jest przetwarzanych przez warstwę. Natomiast liczba kanałów wyjściowych określa, ile różnych wektorów wyjściowych jest generowanych przez warstwę. Liczba wag, dla danej warstwy jest równa $kernel_size \times in_channels \times out_channels$

3 ResNet18

Jednym z największych problemów w uczeniu sieci konwolucyjnych jest problem znikającego gradientu (ang. vanishing gradient). Polega on na tym, że podczas uczenia sieci, propagowane gradienty stają się coraz mniejsze, co prowadzi do tego, że w końcu stają się zbyt małe, aby mogły skutecznie aktualizować wagi sieci. Problem ten zapobiega budowaniu naprawdę głębokich sieci konwolucyjnych, które są w stanie uczyć się złożonych reprezentacji danych.

Rozwiązanie tego problemu zostało zaproponowane w pracy [1]. Autorzy zaproponowali architekturę sieci, w której podzielili sieć na bloki, które mają połączenia rezydualne (wektor wejściowy jest dodawany do wektora wyjściowego bloku). Połączenia te pozwalają na propagowanie gradientów przez sieć, co zapobiega problemowi jego zanikania. Znane są różne warianty tej architektury, które różnią się liczbą bloków oraz liczbą warstw (co można zaobserwować na Rysunku 2).

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2.x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3.x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4.x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5.x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Rysunek 2: Różne warianty architektury ResNet.

W naszym projekcie wykorzystaliśmy wariant ResNet18, ponieważ jest on stosunkowo mały, co pozwala na szybsze uczenie, a jednocześnie zapobiega przeczaczeniu modelu. Przyjrzyjmy się bliżej tej architekturze. Na początku sieci znajduje się warstwa konwolucyjna, która służy do wstępnego przetworzenia danych wejściowych. Następnie znajduje się warstwa MaxPooling, która służy do zmniejszenia rozmiaru danych. Po niej znajdują się 4 bloki, które składają się z dwóch warstw konwolucyjnych wraz z batch normalization oraz funkcją aktywacji ReLU. W każdej z tych warstw znajduje się wspomniane wcześniej połączenie rezydualne. Na samym końcu sieci znajduje się warstwa global average pooling, która służy do zamiany wektora z każdego kanału na pojedynczą wartość, a następnie warstwa gęsta, która będzie służyła do klasyfikacji danych (w naszym przypadku mamy tylko jedną klasę, bo jest to problem binarnej klasyfikacji).

Literatura

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [2] Simon JD Prince. *Understanding deep learning*. MIT press, 2023.